

Python for Data Science

Buổi 7: Time Series & Pandas Hiệu Năng Cao

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 7

- 1 Ôn tập Buổi 3–6 và đặt vấn đề
- 2 Module 25: Vectorized String Operations
- 3 Module 26: Ngày giờ trong Python & Pandas
- 4 Module 27: Time Series – `date_range`, `resample`, `shift`, `rolling`
- 5 Module 28: `eval()` & `query()` – Hiệu năng cao
- 6 Bài tập áp dụng: MovieLens, `births.csv`

Lưu ý phạm vi

Buổi 7 khép lại phần "Pandas cốt lõi" bằng hai kiểu dữ liệu đặc thù (chuỗi, thời gian) và kỹ thuật tối ưu hiệu năng – bước chuẩn bị cuối cùng trước khi bước sang Trực quan hóa dữ liệu (Buổi 8–9).

Nhắc lại Buổi 3–6

- Buổi 3–4 – Series/DataFrame, đọc dữ liệu từ nhiều định dạng.
- Buổi 5 – Missing data, MultiIndex, concat, merge/join.
- Buổi 6 – Data cleaning (duplicate, outlier), groupby (Split-Apply-Combine), pivot_table.

Hai điều còn thiếu

- ❶ Tất cả ví dụ từ trước đến nay xử lý số và chuỗi như dữ liệu "phẳng" – nhưng **văn bản** (tên, tiêu đề, mô tả) và **thời gian** (ngày sinh, ngày giao dịch) có những thao tác đặc thù riêng.
- ❷ Khi dữ liệu đủ lớn, cách viết code "tự nhiên" có thể chậm và tốn bộ nhớ hơn cần thiết.

Buổi 7 trả lời trực tiếp cả hai vấn đề này.

Module 25: Vectorized String Operations

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích vì sao cần accessor `.str` thay vì xử lý chuỗi bằng vòng lặp Python thuần.
- 2 Sử dụng các phương thức `.str` tương tự chuỗi Python (`lower`, `len`, `split`...).
- 3 Áp dụng biểu thức chính quy (regex) qua `.str.extract()`, `.str.findall()`.
- 4 Trích xuất biến chỉ báo (indicator variable) từ dữ liệu multi-label bằng `.str.get_dummies()`.

Vì sao cần accessor .str?

```
data = ['peter', 'Paul', None, 'MARY', 'gUIDO']

[s if s is None else s.capitalize() for s in data] # cách thủ công: dài dòng, dễ lỗi
# ['Peter', 'Paul', None, 'Mary', 'Guido']

names = pd.Series(data)
names.str.capitalize() # cách Pandas: 1 dòng, TỰ ĐỘNG bỏ qua None
# 0    Peter
# 1    Paul
# 2     NaN    <- None được xử lý đúng, không báo lỗi
# 3    Mary
# 4    Guido
```

Đối với mảng số, NumPy/Pandas cho phép *vector hóa* phép toán (Buổi 2). Với mảng chuỗi, Python thuần buộc phải viết vòng lặp và tự kiểm tra None thủ công. Accessor .str của Series mang lại cú pháp vector hóa và tự động xử lý dữ liệu thiếu.

Các phương thức tương tự chuỗi Python

```
monte = pd.Series(["Graham Chapman", "John Cleese", "Terry Gilliam",
                  "Eric Idle", "Terry Jones", "Michael Palin"])

print(monte.str.lower())          # áp dụng .lower() cho TUNG phần tử
print(monte.str.len())           # đo dài từng chuỗi -> trả về số
print(monte.str.startswith("T")) # -> trả về boolean
print(monte.str.split())         # -> trả về list ["Graham","Chapman"]...
```

Gần như toàn bộ phương thức chuỗi có sẵn của Python (lower, upper, strip, replace, contains, split...) đều có phiên bản vector hóa tương ứng qua `.str.<tên_hàm>()`, áp dụng đồng loạt cho mọi phần tử của Series.

Biểu thức chính quy (regex) với .str

```
print(monte.str.extract("([A-Za-z]+)", expand=False))    # lay TEN dau tien
# 0      Graham
# 1      John
# ...

print(monte.str.findall(r"^[^AEIOU].*[^aeiou]\$"))      # ten bat dau/ket thuc = phu am
# 2      [Terry Gilliam]      <- "Terry Gilliam": T (phu am) ... m (phu am)
# 4      [Terry Jones]
# ...      (cac dong khac: list rong [])
```

.str.extract() và .str.findall() (cùng match, contains, replace...) áp dụng cú pháp regex chuẩn của module re lên toàn bộ Series – công cụ mạnh nhất để làm sạch văn bản lộn xộn từ nguồn thực tế (web scraping, khảo sát...).

Truy cập theo vị trí & biến chỉ báo (indicator)

```
print(monte.str[0:3])                # 3 ký tự đầu -- như slicing Python
print(monte.str.split().str[-1])      # họ (tên cuối) của từng người

full_monte = pd.DataFrame({"name": monte,
                           "info": ["B|C|D", "B|D", "A|C", "B|D", "B|C", "B|C|D"]}) # B=UK, C=Canada, D=..
print(full_monte["info"].str.get_dummies("|"))
#      A  B  C  D
# 0  0  0  1  1  1
# ...
```

`.str[i]/.str.slice()` cho phép "chỉ mục hóa" từng phần tử chuỗi; kết hợp với `.str.split()` có thể trích xuất họ/tên. `.str.get_dummies(sep)` (đã dùng với thể loại phim ở Buổi 6) tổng quát cho *mọi* cột chứa mã nhiều nhãn phân tách bởi 1 ký tự.

Tổng kết Module 25

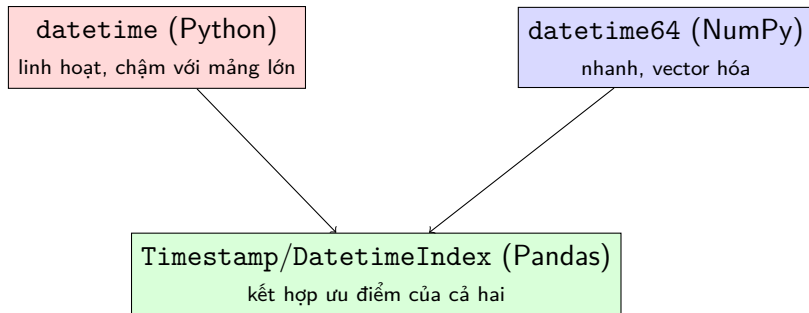
- `.str`: accessor vector hóa thao tác chuỗi, tự động xử lý None/NaN.
- Phần lớn phương thức chuỗi Python có bản sao chép `.str.<tên>()`.
- `.str.extract()`, `.str.findall()`, `.str.contains()`: dùng regex để trích xuất/lọc.
- `.str[i]`, `.str.get_dummies(sep)`: truy cập vị trí, mã hóa dữ liệu multi-label.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân biệt 3 tầng biểu diễn thời gian: `datetime` (Python), `datetime64` (NumPy), `Timestamp` (Pandas).
- 2 Chuyển đổi linh hoạt sang thời gian bằng `pd.to_datetime()`.
- 3 Dùng `DatetimeIndex` làm chỉ mục để lọc/slicing dữ liệu theo thời gian.

Ba tầng biểu diễn thời gian



Pandas không phát minh lại: `Timestamp` kế thừa sự tiện lợi của `datetime/dateutil` và tốc độ lưu trữ nhị phân của `numpy.datetime64` – "tốt nhất của cả hai thế giới".

pd.to_datetime(): linh hoạt phân tích nhiều định dạng

```
from datetime import datetime

dates = pd.to_datetime([datetime(2021, 7, 3), "4th of July, 2021",
                        "2021-Jul-6", "07-07-2021", "20210708"])

print(dates)
# DatetimeIndex(['2021-07-03', '2021-07-04', '2021-07-06', '2021-07-07', '2021-07-08'])

print(dates.to_period("D"))          # DatetimeIndex -> PeriodIndex (khoảng thời gian)
print(dates - dates[0])              # phép trừ -> TimedeltaIndex
```

pd.to_datetime() tự nhận diện *nhiều định dạng ngày tháng khác nhau* trong cùng một lệnh – rất hữu ích khi làm sạch dữ liệu thời gian không đồng nhất từ nhiều nguồn (khảo sát, log hệ thống...).

DatetimeIndex: lọc dữ liệu theo thời gian

```
index = pd.DatetimeIndex(["2020-07-04", "2020-08-04", "2021-07-04", "2021-08-04"])
data = pd.Series([0, 1, 2, 3], index=index)
```

```
print(data["2020-07-04":"2021-07-04"])    # slicing theo NGÀY -- nhu index thuong
# 2020-07-04    0
# 2020-08-04    1
# 2021-07-04    2
```

```
print(data["2021"])                        # slicing theo NAM -- cu phap dac biet
# 2021-07-04    2
# 2021-08-04    3
```

Khi `DatetimeIndex` là chỉ mục, Pandas hỗ trợ cú pháp "slicing thời gian một phần" (partial-string indexing): chỉ cần truyền năm ("2021") hoặc năm-tháng ("2021-07") để lấy toàn bộ dữ liệu trong khoảng đó.

Tổng kết Module 26

- 3 tầng: `datetime` (linh hoạt) $\rightarrow$ `datetime64` (nhanh) $\rightarrow$ `Timestamp/DatetimeIndex` (kết hợp cả hai).
- `pd.to_datetime()`: chuyển đổi linh hoạt từ nhiều định dạng chuỗi/đối tượng ngày tháng.
- `DatetimeIndex - DatetimeIndex  $\rightarrow$  TimedeltaIndex`; `.to_period()` $\rightarrow$ `PeriodIndex`.
- `DatetimeIndex` làm chỉ mục hỗ trợ slicing theo ngày, tháng, hoặc năm trực tiếp.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Tạo chuỗi thời gian đều đặn bằng `pd.date_range()` và các mã tần suất (frequency code).
- 2 Phân biệt `resample()` (tổng hợp) và `asfreq()` (chọn giá trị) khi đổi tần suất.
- 3 Dùng `shift()` để so sánh dữ liệu qua các mốc thời gian.
- 4 Dùng `rolling()` để tính thống kê theo cửa sổ trượt.

Tạo chuỗi thời gian đều đặn: date_range

```
print(pd.date_range("2015-07-03", "2015-07-10"))          # tu ngay den ngay, mac dinh D
print(pd.date_range("2015-07-03", periods=8))              # tu ngay + so ky nguyen
print(pd.date_range("2015-07-03", periods=8, freq="h"))     # tan suat GIO
print(pd.period_range("2015-07", periods=8, freq="M"))      # 8 THANG lien tiep
print(pd.timedelta_range(0, periods=6, freq="2h30min"))     # khoang cach 2h30p
```

Tương tự `np.arange()` (Buổi 2) nhưng cho thời gian: `date_range` (mốc thời gian), `period_range` (khoảng thời gian), `timedelta_range` (độ dài thời gian) – đều nhận `start/end/periods` và tham số `freq`.

Một số mã tần suất (frequency code) thường dùng

Mã	Ý nghĩa	Mã	Ý nghĩa
D	Ngày theo lịch	B	Ngày làm việc
W	Tuần	ME	Cuối tháng
MS	Đầu tháng	QE	Cuối quý
YE	Cuối năm	h	Giờ
min	Phút	s	Giây

Có thể kết hợp số nhân với mã (ví dụ "2h30min" = 2 giờ 30 phút); truy cập trực tiếp offset qua `pandas.tseries.offsets` (ví dụ `BDay()` cho ngày làm việc).

resample() vs asfreq(): tổng hợp hay chọn giá trị?

```
total.resample("W").sum()      # TONG HOP: tong so ca sinh trong TUNG TUAN
total.resample("ME").mean()    # TONG HOP: trung binh so ca sinh MOI NGAY, theo thang

total.asfreq("W")              # CHON GIA TRI: lay dung 1 gia tri tai moc cuoi tuan
```

Khác biệt cốt lõi

`resample()` về bản chất là một phép **tổng hợp dữ liệu** (aggregation) – giống `groupby()` nhưng nhóm theo thời gian; cần chỉ định hàm tổng hợp (`.sum()`, `.mean()`...). `asfreq()` chỉ đơn thuần **chọn lại** giá trị tại mốc thời gian mới, không tính toán gì thêm – nếu không có dữ liệu đúng mốc đó, kết quả là NaN.

Ứng dụng thật: sổ ca sinh tại Hoa Kỳ (births.csv)

```
births = pd.read_csv("notebooks/data/births.csv")    # kho PythonDataScienceHandbook
births = births.dropna(subset=["day"])
births = births[births["day"].between(1, 31)]        # loại ngày lỗi như "99"
dates = pd.to_datetime(dict(year=births.year, month=births.month, day=births.day),
                        errors="coerce")
births = births[dates.notna()]
births.index = dates[dates.notna()]

total = births.groupby(births.index)["births"].sum()    # tong nam+nu theo NGÀY
print(total.resample("W").sum().head())                # 44380, 66136, 67664, 67008, 66632...
```

Dữ liệu thật (CDC, 1969–1988, 14 610 ngày hợp lệ sau khi loại bỏ ngày lỗi kiểu "June 99th" – một dạng làm sạch dữ liệu đã học ở Module 21, Buổi 6) cho thấy quy trình đầy đủ: đọc CSV → dựng DatetimeIndex → groupby theo ngày → resample theo tuần.

shift() và rolling(): so sánh theo thời gian, làm mượt dữ liệu

```
diff_week = total - total.shift(7)          # so sanh MOI NGAY voi CUNG NGAY tuan truoc
print(diff_week.dropna().head())
# 1969-01-08    1248.0
# 1969-01-09     432.0    (so ca sinh ngay 9/1 - so ca sinh ngay 2/1)

rolling_mean = total.rolling(30, center=True).mean()  # trung binh truot 30 ngay
print(rolling_mean.iloc[200])          # ~10214.9 -- da lam MUOT bien dong tung ngay
```

shift(k) dịch toàn bộ dữ liệu đi k vị trí – kết hợp phép trừ cho ra "biến động so với k kỳ trước" (ứng dụng kinh điển: tỉ suất sinh lời tài chính). rolling(w) tính thống kê trên cửa sổ trượt độ rộng w – center=True căn giữa cửa sổ, làm mượt nhiễu ngày-quá-ngày để lộ ra xu hướng dài hạn.

Kết hợp với groupby: mẫu hình theo ngày trong tuần

```
by_dow = total.groupby(total.index.dayofweek).mean() # 0=T2 ... 6=CN
by_dow.index = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
print(by_dow.round(1))
# Mon 9944.2   Tue 10368.0   Wed 10089.5   Thu 9995.0
# Fri 10146.8   Sat 8708.3   Sun 8290.5
```

DatetimeIndex có các thuộc tính có sẵn (.dayofweek, .month, .year...) dùng trực tiếp làm khóa groupby() (Buổi 6). **Kết quả kiểm chứng thực tế:** số ca sinh vào cuối tuần (Thứ Bảy, Chủ Nhật) thấp hơn rõ rệt so với ngày thường – phần lớn do lịch sinh mổ/khởi phát chuyển dạ theo kế hoạch của bệnh viện.

Tổng kết Module 27

- `pd.date_range/period_range/timedelta_range`: tạo chuỗi thời gian đều đặn theo `freq`.
- `resample()`: tổng hợp (aggregation) theo tần suất mới; `asfreq()`: chỉ chọn lại giá trị.
- `shift(k)`: dịch dữ liệu k kỳ – dùng để tính biến động/so sánh theo thời gian.
- `rolling(w)`: thống kê theo cửa sổ trượt – làm mượt, lộ xu hướng dài hạn.
- Thuộc tính `DatetimeIndex (.dayofweek...)` kết hợp tốt với `groupby()`.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích vì sao biểu thức phức hợp trên mảng lớn tốn bộ nhớ/thời gian.
- 2 Sử dụng `pd.eval()/DataFrame.eval()` để tính toán và gán cột hiệu quả hơn.
- 3 Sử dụng `DataFrame.query()` để lọc dữ liệu dễ đọc và nhanh hơn.
- 4 Đánh giá khi nào `eval/query` thực sự mang lại lợi ích hiệu năng.

Vấn đề: biểu thức phức hợp tạo nhiều mảng tạm

```
mask = (x > 0.5) & (y < 0.5)

# ve ban chat, Pandas/NumPy thuc hien TUNG BUOC rieng le:
tmp1 = (x > 0.5)      # cap phat 1 mang tam
tmp2 = (y < 0.5)      # cap phat 1 mang tam nua
mask = tmp1 & tmp2    # cap phat mang ket qua thu 3
```

Mỗi phép toán con trong 1 biểu thức tạo ra một mảng *trung gian* được cấp phát bộ nhớ riêng, dù ta không bao giờ dùng đến nó sau đó. Với DataFrame lớn (hàng trăm MB trở lên), tổng chi phí cấp phát/giải phóng các mảng tạm này trở nên đáng kể.

pd.eval(): tính toán hiệu quả bằng biểu thức chuỗi

```
df1, df2, df3, df4 = (pd.DataFrame(rng.random((500_000, 100))) for _ in range(4))
# moi DataFrame ~400 MB

result_thuong = df1 + df2 + df3 + df4
result_eval = pd.eval("df1 + df2 + df3 + df4")      # ket qua GIONG HET, engine khac
np.allclose(result_thuong, result_eval)            # True

pd.eval("df1 < df2 <= df3 != df4")                # ho tro toan tu so sanh, "and"/"or", @bien...
```

Kết quả đo thực tế (sandbox, 2 lõi CPU): với 4 DataFrame $\approx$ 400 MB, `pd.eval()` nhanh hơn $\approx$ 1,5 lần so với cộng trực tiếp nhờ tránh cấp phát mảng tạm (thư viện `numexpr` thực thi biểu thức theo từng khối nhỏ, tận dụng cache CPU).

DataFrame.eval(): cột như biến, gán cột mới

```
df = pd.DataFrame(rng.random((1000, 3)), columns=["A", "B", "C"])

result = df.eval("(A + B) / (C - 1)")      # ten cot dung TRUC TIEP nhu bien Python

df.eval("D = (A + B) / C", inplace=True)    # TAO cot moi D ngay trong bieu thuc

column_mean = df.mean(axis=1)
df.eval("E = A + @column_mean", inplace=True)  # @ danh dau BIEN PYTHON, khong phai cot
```

`DataFrame.eval()` coi tên cột như tên biến – ngắn gọn hơn nhiều so với `df["A"]`. Ký hiệu `@` phân biệt rõ ràng "không gian tên" của *cột* và của *biến Python cục bộ* khi cả hai cùng xuất hiện trong 1 biểu thức.

DataFrame.query(): lọc dữ liệu dễ đọc hơn

```
result1 = df[(df.A < 0.5) & (df.B < 0.5)]      # cách truyền thông: nhiều dấu ngoặc
result2 = df.query("A < 0.5 and B < 0.5")      # cách query: đọc như câu văn

Cmean = df["C"].mean()

result3 = df.query("A < @Cmean and B < @Cmean") # @ cho biến Python, như eval()
```

`query()` *không thể* biểu diễn được bằng `DataFrame.eval()` thông thường (vì kết quả là lọc dòng, không phải phép tính trên cột) – đây là phương thức chuyên biệt cho lọc dữ liệu, đã dùng ở Buổi 6 (`spice_df.query(...)`) mà chưa giải thích cơ chế.

Khi nào nên dùng `eval()`/`query()`?

Hai tiêu chí

- ❶ **Bộ nhớ:** nếu kích thước dữ liệu tạm ($\approx$ vài lần kích thước DataFrame) so sánh được với RAM khả dụng – kiểm tra bằng `df.values.nbytes`.
- ❷ **Cache CPU:** lợi ích rõ nhất khi dữ liệu lớn hơn nhiều so với cache L1/L2 (thường vài MB) – với dữ liệu nhỏ, chi phí khởi tạo `numexpr` có thể khiến `eval/query` *chậm hơn* cách viết thông thường.

Bài học về đo lường hiệu năng

Tốc độ thực tế phụ thuộc phần cứng (số lõi CPU, kích thước cache, kích thước dữ liệu) – **luôn đo trên máy của chính mình** bằng module `timeit` thay vì tin tưởng tuyệt đối vào một con số cụ thể đọc được từ sách/slide.

Tổng kết Module 28

- Biểu thức phức hợp tạo nhiều mảng tạm – tốn bộ nhớ & thời gian với dữ liệu lớn.
- `pd.eval()/DataFrame.eval()`: tính toán qua `numexpr`, tránh mảng tạm; hỗ trợ gán cột (`"D = ..."`).
- `DataFrame.query()`: lọc dòng, cú pháp đọc tự nhiên hơn boolean mask lồng nhau.
- `@bien`: phân biệt biến Python cục bộ với tên cột trong biểu thức.
- Lợi ích thực sự phụ thuộc kích thước dữ liệu & phần cứng – luôn benchmark, không giả định.

Ví dụ đầy đủ: trích xuất năm sản xuất từ tên phim (MovieLens)

```
mnames = ["movie_id", "title", "genres"]
movies = pd.read_table("datasets/movielens/movies.dat", sep="::", header=None,
                      names=mnames, engine="python", encoding="latin-1")

years = movies["title"].str.extract(r"\((\d{4})\)")      # lay 4 chu so trong ngoac
movies["year"] = years[0]
print(movies[["title", "year"]].head(2))
#           title  year
# 0  Toy Story (1995)  1995
# 1    Jumanji (1995)  1995

print(movies["year"].isna().sum())      # 0 -- toan bo 3883 phim deu khop duoc
```

Toàn bộ 3883 phim trong MovieLens đều có năm sản xuất trong ngoặc đơn ở cuối tiêu đề – một quy ước dữ liệu giúp `.str.extract()` với 1 dòng regex trích xuất chính xác 100%, biến cột văn bản thành cột số dùng được cho `groupby/pivot_table` (Buổi 6).

Bài tập 1: Mở rộng phân tích văn bản MovieLens

Dữ liệu: `datasets/movielens/movies.dat` (đã dùng ở Buổi 6 & trang trước).

- ❶ Dùng cột `year` vừa trích xuất, kết hợp `pd.cut()` (Buổi 6) chia phim thành các thập kỷ; đếm số phim theo từng thập kỷ bằng `groupby`.
- ❷ Dùng `movies["title"].str.replace(r"\\(\\d{4}\\)", "", regex=True).str.strip()` để lấy tên phim *không kèm năm* – kiểm tra xem có phim nào bị trùng tên (nhưng khác năm sản xuất) không.
- ❸ Dùng `.str.contains("War", case=False)` để tìm tất cả phim có chữ "War" trong tiêu đề; kết hợp `.str.extract()` lấy năm để xem các phim này trải dài qua những thập kỷ nào.
- ❹ *Câu hỏi mở*: nếu muốn dùng `query()` thay vì boolean mask cho bài tập 3, cần chuẩn bị cột nào trước? Vì sao `query()` không áp dụng trực tiếp lên kết quả của `.str.contains()`?

Bài tập 2: Time series mở rộng trên births.csv

Dữ liệu: `notebooks/data/births.csv` (đã dùng ở các trang Module 27).

- 1 Tính `total.resample("YE").sum()` – tổng số ca sinh theo từng năm (1969–1988). Vẽ nhận xét: xu hướng tăng/giảm qua các năm có đơn điệu không?
- 2 So sánh `rolling(7).mean()` (trung bình tuần) với `rolling(30).mean()` (trung bình tháng) trên cùng 1 đoạn dữ liệu 2 năm bất kỳ – đường nào mượt hơn? Đánh đổi giữa độ mượt và độ trễ phản ánh xu hướng là gì?
- 3 Dùng `query()` (Module 28) để lọc riêng dữ liệu của năm 1980, sau đó tính lại `groupby(index.dayofweek)` cho riêng năm này – mẫu hình cuối tuần có còn rõ rệt như tính trên toàn bộ 1969–1988 không?
- 4 Dùng `shift(365)` (thay vì `shift(7)`) để tính chênh lệch số ca sinh so với *đúng ngày này năm trước* – kết quả có ổn định quanh 0 hay có xu hướng tăng dần? Giải thích bằng dân số Hoa Kỳ tăng theo thời gian.

Bảng tổng hợp các thao tác Pandas trong Buổi 7

Nhóm	Hàm/thao tác	Mục đích
Chuỗi	<code>.str.<method></code> , <code>.str.extract</code> , <code>.str.findall</code>	Vector hóa xử lý/trích xuất văn bản
Chuỗi	<code>.str[i]</code> , <code>.str.get_dummies(sep)</code>	Truy cập vị trí, mã hóa multi-label
Thời gian	<code>pd.to_datetime</code> , <code>DatetimeIndex</code>	Chuyển đổi & lập chỉ mục theo thời gian
Chuỗi thời gian	<code>date_range</code> , <code>period_range</code>	Tạo mốc/khoảng thời gian đều đặn
Tổng hợp thời gian	<code>resample</code> , <code>asfreq</code>	Đổi tần suất (tổng hợp / chọn giá trị)
So sánh thời gian	<code>shift</code> , <code>rolling</code>	Độ trễ, cửa sổ trượt
Hiệu năng	<code>pd.eval</code> , <code>DataFrame.eval</code> , <code>.query</code>	Tránh mảng tạm, cú pháp lọc/tính gọn hơn

Tài liệu tham khảo I

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – notebooks 03.10 (Vectorized String Operations), 03.11 (Working with Time Series), 03.12 (High-Performance Pandas: eval and query); dữ liệu births.csv]. GitHub. <https://github.com/jakevdp/PythonDataScienceHandbook>
- Pugh, D. R. (n.d.). *python-for-data-analysis* [Kho mã nguồn – dữ liệu movielens]. GitHub. <https://github.com/davidrpugh/python-for-data-analysis>
- Centers for Disease Control and Prevention (CDC). *U.S. births by date* [Bộ dữ liệu]. (dẫn qua notebook 03.09/03.11 của PythonDataScienceHandbook).
- GroupLens Research. *MovieLens 1M Dataset*. University of Minnesota. <https://grouplens.org/datasets/movielens/1m/>

- The pandas development team. (2020). *pandas-dev/pandas: Pandas (Version 1.0) [Software]*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Chuẩn bị cho buổi tiếp theo: Trực Quan Hóa Dữ Liệu I – Matplotlib Nền Tảng

Thông điệp

Buổi 7 khép lại toàn bộ phần "Pandas cốt lõi" (Buổi 3–7): đọc dữ liệu, làm sạch, kết hợp, tổng hợp, xử lý chuỗi & thời gian. Từ Buổi 8, trọng tâm chuyển sang *trình bày* dữ liệu một cách trực quan – kỹ năng bắt buộc trước khi thực hành EDA hoàn chỉnh ở Buổi 9 và bước sang Machine Learning từ Buổi 10.